

Agents as Processes

What a language model is, in this class's terms. Sandboxing is virtualization. The loops are closed. The gate is what you built.

[← SE 12 · Universality for Systems](#)

WHAT IT IS

A function from text to a distribution over the next token, with 10^{11} parameters, run in a loop.

Trained on most of the written record to predict the next piece of text; sampled one token at a time and fed its own output back. Nothing in it is a semantics: it does not run programs, check proofs, or consult the world.

That makes it superb at everything that is a property of notation: fluency, style, translation. And structurally unable to certify a property of meaning. A hallucination is a proof-shaped object that is not true, and the machine cannot tell.

IN SYSTEMS TERMS

A program on someone's hardware, consuming a great deal of energy, whose output is other programs, commands and requests. Untrusted by construction: not malicious, but unverified, and this class is about what a kernel does with those.

SANDBOXING IS VIRTUALIZATION

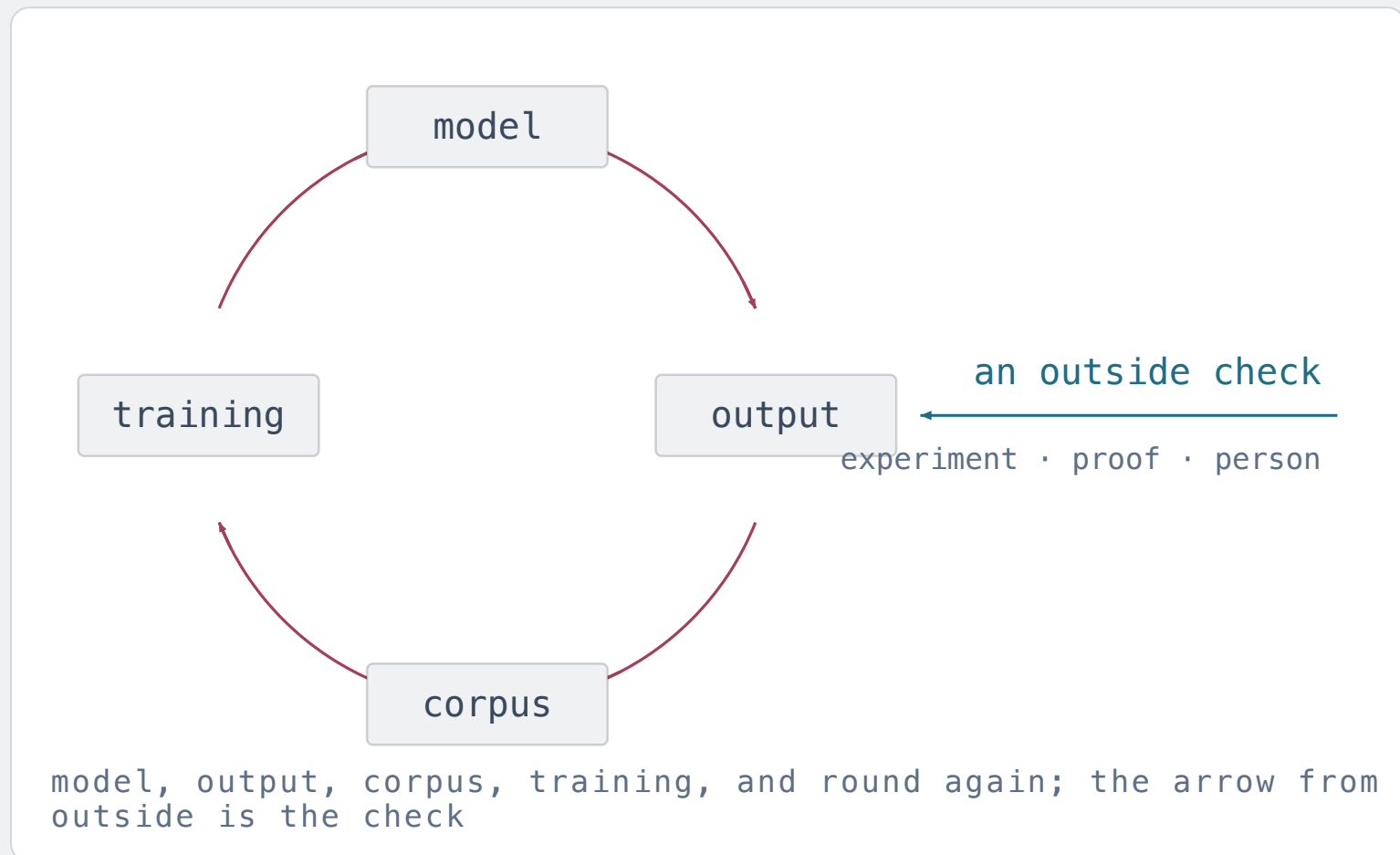
An agent runs generated code. Everything you built is what runs it safely.

THE AGENT NEEDS	THE KERNEL PROVIDES	WEEK
run code it did not verify	a process, isolated in space by paging	4, 7
code that may not stop	a timer; a bound in instructions, seconds, or tokens	5
a limited view of the world	the door: system calls, and the kernel says no	2, 7
many attempts, in parallel	fork; cheap contexts; copy-on-write	7
a budget	the profile: instructions, memory, joules	11
trust in the sandbox itself	a small, named TCB; a kernel beneath the kernel	6, 12

The sandbox around an agent is a virtual machine, and its designer faces the self-reference of week 6: the agent may try to reach the sandbox's controls, and the isolation of the controls is the whole question. Nothing new; harder stakes.

THE LOOPS ARE CLOSED

Models train on model output. Models grade models. Agents invoke themselves. And no system can **certify itself**.

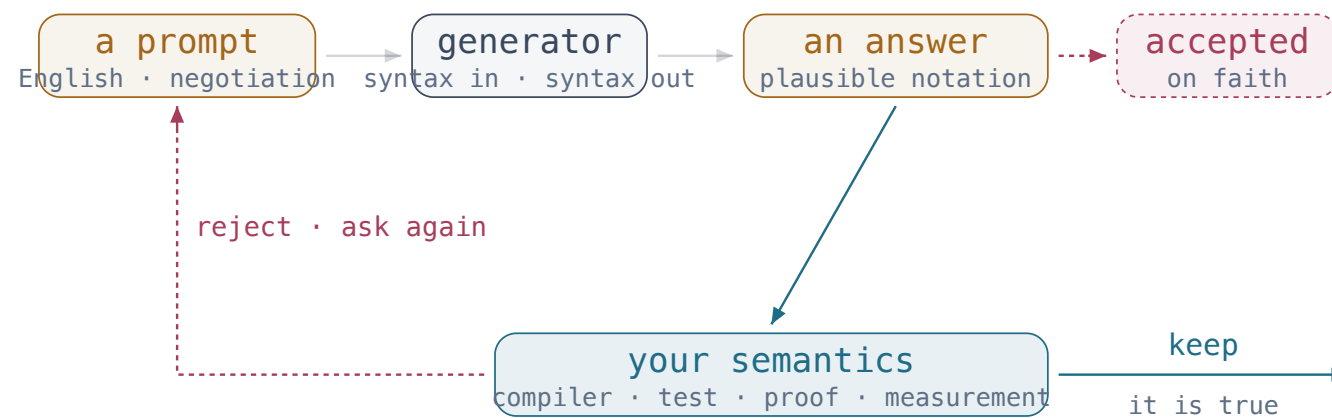


Self-reference in the machines: model collapse when the loop tightens, judges sharing the candidate's blind spots, agents calling agents.

Is this system safe? A semantic property: Rice says no general procedure. Can it explain itself? By Gödel II an explanation is more output, not a certificate. The kernel never asks the process; it checks from outside.

So the effort goes where the arrow enters from outside: the gate.

Ask a machine for a systems routine. Then **mipster**, your kernel, **rotor**.



a generator supplies notation; the gate supplies the semantics

the only part the machine does not supply

```

$ ./selfie -c generated.c -m 1
./selfie: ... exception: invalid memory access at 0x...
$ ./selfie -c selfie.c -m 2 -c generated.c -y 1
$ ./rotor -c generated.c - 0
$ tools/bitme.py -kmax 1000 generated-rotorized.btor2
... core-0-load-seg-fault - satisfiable · input ...
  
```

Does it run? On mipster, isolated: the exception handler catches what the generator did not. Under your kernel? Under hypster, time-shared, with your locks. Can it fail? Within a thousand steps, on any input: absence within a bound.

None of the three is available to the machine that wrote it: the check the generator cannot be. One theorem, fourth appearance, and this class built all three checks.

WHAT CHANGED

Generation got cheap. Isolation, bounding and verification **did not**.

Before, producing a program was expensive and scarcity did the filtering. Now production is nearly free and unbounded, and the filter has to be built: a kernel to run it in, a bound to stop it, a check to accept it. Hard to find, easy to check, at consumer scale.

Value migrates to the ends the machine does not occupy: stating what should be true, and establishing that it is. For a systems engineer the second is the job description. Own the ends.

BEFORE NEXT WEEK

The Machines chapter's exercises 1, 2 and 6, with a systems routine instead of factorial. Bring the failing input rotor found, or the bound up to which none exists. Next week: what a system is.

Listen, then read.

LISTEN

[Strauss · Salome](#) — Karajan, Vienna 1978. Herod grants a capability without a gate: whatever you ask, up to half my kingdom. Salome asks. An agent given more than its sandbox should allow, and the opera is the consequence, in ninety minutes.

READ

[Human Compatible](#) — Russell, 2019: machines that pursue objectives they were given, and why the sandbox around them is the design problem. And, more technical, [A Note on the Confinement Problem](#) — Lampson, 1973: can a program be prevented from leaking what it was given? Three pages, and the answer is still no.